

A Compositional Mathematics of Optimization Fields

Categorical Semantics, Probabilistic Evaluation, Lawful Interventions, and a Minimal Plugin Kernel

Architecture Study for RAG-MATHS

August 2026

Contents

Abstract	4
1. Introduction	5
1.1 The architectural problem	5
1.2 Why category theory is useful here	6
1.3 Thesis statement	6
2. Requirements of an Optimization Field	7
2.1 Required invariants	7
3. A Free Typed Process Category	8
3.1 Objects as ports	8
3.2 Composition	8
3.3 Free syntax and interpreters	9
3.4 Cost as a monoidal interpretation	9
3.5 Effects as static annotations	11
3.6 Dependency labels	11
4. Plugins as Algebraic Signatures	11
4.1 Why conventional plugin callbacks are too weak	11
4.2 A plugin does not own composition	12
4.3 Plugin laws	12
4.4 Capability-oriented extension	12
4.5 Process isolation	13
5. Parameterized Morphisms and the Para Construction	13
5.1 Systems vary with parameters	13
5.2 Reparameterization	13
5.3 Optimization spaces are not necessarily Euclidean	14
6. Optics and Lawful Interventions	14
6.1 Local change as a first-class object	14
6.2 Why optics matter to experimental validity	15

6.3 Composition of lenses	15
6.4 Beyond lenses	15
7. Probability Kernels and Experimental Semantics	16
7.1 Evaluation is a stochastic map	16
7.2 Deterministic seed splitting	16
7.3 Coupling baseline and candidate	17
7.4 Distributional equality and observational equivalence	17
7.5 Why the generic kernel should not own statistical doctrine	17
8. Metrics, Ordered Decisions, and Open Games	18
8.1 Metrics form observations, not utility by default	18
8.2 Ordered gates	18
8.3 Open-game intuition	18
9. Campaigns as Coalgebraic Feedback Processes	19
9.1 Campaign state	19
9.2 Proposal strategies are plugins over evidence	19
9.3 Trial coordinates	20
9.4 Append-only evidence	20
9.5 Termination	20
10. The Minimal Software Kernel	20
10.1 Kernel responsibilities	20
10.2 What is intentionally absent	21
10.3 Typed envelopes and identity	21
10.4 Descriptors	21
10.5 Static inspection before execution	22
11. RAG as a Grafted Optimization Domain	22
11.1 The point of the example	22
11.2 Domain types	22
11.3 The RAG operation as a primitive generator	22
11.4 Candidate spaces	24
11.5 Evaluation	24
11.6 From ragopt to the proposed backbone	24
12. Plugin Architecture for Real Systems	25
12.1 Four plugin strata	25
Operation plugins	25
Domain plugins	25
Evaluation plugins	25
Proposal plugins	25
12.2 Interface sketch	25
12.3 Manifested semantics	26
12.4 Host control	26
13. Correctness and Proof Obligations	27
13.1 Composition laws	27

13.2 Interpreter coherence	27
13.3 Lens laws	27
13.4 Seed and pairing laws	27
13.5 Evidence laws	27
13.6 Decision laws	28
14. Relationship to Existing Mathematical Work	28
14.1 Monads and effectful computation	28
14.2 Arrows and generalized computation	28
14.3 Parameterized maps and learning	28
14.4 Optics	28
14.5 Markov categories	28
14.6 Open games and compositional optimization	28
14.7 Differentiable programming	29
15. A More General Theory of Optimization Morphisms	29
15.1 Optimization as an outer category	29
15.2 Fibred view of parameter spaces	29
15.3 Multi-fidelity evaluation as morphisms between observation spaces . .	30
16. From Mathematical Core to Production Architecture	30
16.1 The core should remain library-sized	30
16.2 Suggested package layering	30
16.3 Relationship to ragkit and ragopt	31
17. Sandbox Implementation	31
17.1 Scope	31
17.2 Running the demo	32
17.3 What the demo proves	32
18. Worked RAG Mapping	32
Fusion intervention	33
Chunking intervention	33
ANN intervention	33
19. Research Agenda	33
19.1 Canonical quotient of plans	33
19.2 Effect capabilities rather than labels	34
19.3 Enriched cost categories	34
19.4 Markov-category evaluation	34
19.5 Optic-indexed dependency analysis	34
19.6 Compositional decision theory	34
19.7 Differentiable subdomains	34
19.8 Formal verification	34
20. Conclusion	35
Appendix A. Core Algebra	35

Appendix B. Plugin Checklist	36
Appendix C. Sandbox File Map	36
Appendix D. Suggested Next Implementation Increments	37
Appendix E. Selected Bibliography	38

Abstract

This thesis develops a mathematical and executable backbone for *composable optimization fields*: systems in which domain-specific artifacts are constructed, perturbed, evaluated under uncertainty, compared, and iteratively improved while preserving enough semantic structure to reason about correctness, provenance, reproducibility, and composition. The motivating application is retrieval-augmented generation (RAG), where an optimization campaign may change indexing, chunking, representations, embeddings, retrieval policy, reranking, generation, or runtime policy. The proposed theory is deliberately more general. It treats RAG as one plugin family grafted onto a small optimization kernel rather than making RAG concepts primitive.

The central construction has five layers. First, a free typed symmetric-monoidal plan language records composition before choosing an execution semantics. Primitive operations are supplied by plugins as typed generators. The same plan can then be interpreted into execution, effect analysis, dependency closure, cost estimates, provenance, or other semantic carriers. Second, parameterized morphisms model systems whose behavior depends on tunable configuration. Lawful optics provide localized interventions into those configurations and make the claim “change exactly this parameter” precise. Third, stochastic evaluation is modeled with probability kernels, deterministic seed splitting, and explicit couplings. This separates the semantics of the system being optimized from the randomness of measurement and permits paired experiments to be expressed as a coupling rather than as an implementation accident. Fourth, metrics and decision policies are treated as algebraic observations over trials: hard constraints, partial orders, Pareto comparisons, and product-specific gates remain outside the generic optimizer. Fifth, a campaign is a feedback process—a coalgebraic state machine that repeatedly proposes interventions, materializes candidates, evaluates them, accumulates evidence, applies a decision algebra, and either terminates or produces a new state.

The architecture yields a strong plugin boundary. A plugin contributes schemas, primitive operations, intervention spaces, laws, and optional evaluators. It does not receive unrestricted access to the campaign engine. The core can therefore inspect the graph of computation and its declared effects before execution. Domain packages retain authority over what their data means, how candidates are legal, what measurements are valid, and what constitutes promotion. This creates an extensibility model closer to an algebraic signature with models and laws than to a conventional callback framework.

A self-contained Go sandbox accompanies the thesis. Its core implements typed envelopes and content identity, operation descriptors, a plugin registry, free plans with

sequential and tensor composition, static plan analysis, a runtime interpreter, lawful lenses, finite probability distributions and deterministic seeds, candidate spaces, trial runners, append-only resumable campaigns, and a toy RAG plugin. The toy RAG plugin optimizes retrieval weights without introducing RAG knowledge into the core. The implementation is intentionally compact: it is an executable semantic specimen designed to make the theory falsifiable and extendable, not a production framework.

The thesis argues that the correct reusable unit for optimization is not a global “optimizer” abstraction. It is a small category of typed computations plus explicit interfaces for parameterization, stochastic observation, intervention, evidence, and decision. This keeps the core simple while giving real systems—RAG, scheduling, compiler tuning, prompt optimization, model selection, index design, or operational policy—a disciplined way to graft in their own semantics.

1. Introduction

1.1 The architectural problem

Optimization infrastructure often begins as a loop:

```
for candidate in candidates:
    result = evaluate(candidate)
select(best(result))
```

This shape is adequate only while the candidate is a scalar and evaluation is a pure, inexpensive function. It fails once optimization touches real systems. A candidate may require rebuilding derived artifacts. Evaluation may have multiple stages and effects. Some comparisons are valid only when the same cases and random conditions are paired. Different metrics have different directions and different safety status. Failed trials must remain in the denominator. The environment may be partially stochastic. Domain rules determine which changes are legal and which measurements matter. Production promotion may require an ordered sequence of hard gates rather than maximizing a score.

RAG makes these difficulties obvious. A change to fusion weight needs no index rebuild; a change to chunking invalidates representations, embeddings, indexes, and evidence labels; a change to reranking may alter remote disclosure; a change to agent policy must be evaluated at session rather than retrieval level. Treating all of these as `map[string]any` hyperparameters destroys causal structure. Treating all evaluation as a single callback hides the semantics that determine reproducibility and correctness.

The design problem is therefore:

Construct a small mathematical and software backbone in which heterogeneous optimization domains can be composed and inspected without forcing their semantics into a monolithic framework.

The desired system must satisfy two apparently conflicting goals.

1. **Strong semantics.** The core should know enough to validate composition, identify plans, reason about effects and dependencies, represent stochastic evaluation, and preserve evidence.
2. **Weak domain assumptions.** The core should not know what a chunk, compiler flag, SQL plan, model prompt, garden product, or scheduling objective means.

The proposed solution is to make the core algebraic. Domain plugins provide *generators* and *models* of a small signature. The kernel owns composition laws. Plugins own domain meaning.

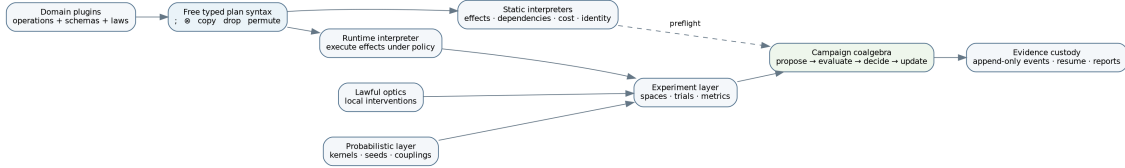


Figure 1: The proposed optimization backbone.

1.2 Why category theory is useful here

Category theory is useful only if it changes the design. Four ideas do.

First, a category separates **things** from **composable transformations**. Typed optimization pipelines naturally have objects—schemas, configurations, artifacts, cases, outcomes—and morphisms between them.

Second, monoidal structure models **independent parallel composition**. An optimizer must distinguish sequential dependency from independent branches because effects, cost, and critical path differ.

Third, free constructions separate **syntax from interpretation**. A plan can be represented once and interpreted many ways. Execution is one interpreter. Effect analysis, dependency analysis, cost projection, provenance, visualization, and verification are others.

Fourth, parameterized morphisms and optics model **controlled change**. Optimization is not ordinary execution. It is the study of families of executions under interventions. The parameter space and the lens by which a parameter is changed are mathematical structure, not incidental configuration parsing.

Probability enters because evaluation is generally not deterministic. A model call, a sampled workload, a production latency, and a human judgment are random variables. The useful abstraction is not “randomness exists,” but a composable stochastic kernel whose coupling can be controlled when comparing candidates.

1.3 Thesis statement

The thesis is:

A reusable optimization architecture can be built from a free typed monoidal process language, lawful parameter optics, stochastic kernels with explicit

coupling, and evidence-preserving campaign state machines. Domain plugins extend the signature and supply laws; the kernel supplies composition and interpreters. This is sufficient to express nontrivial RAG optimization while keeping the core domain-neutral.

The claim is not that every optimizer should expose category-theoretic terminology. The user-facing API can remain ordinary Go. The claim is that these structures provide the right *design invariants*.

2. Requirements of an Optimization Field

An optimization field is the total environment in which interventions can be proposed, realized, observed, compared, and accepted. It has more structure than a search space.

We write a field informally as

$$\mathfrak{F} = (\mathcal{C}, \mathcal{P}, \mathcal{I}, \mathcal{W}, \mathcal{E}, \mathcal{M}, \mathcal{D}),$$

where:

- $\mathcal{C}$ is a category of executable computations;
- $\mathcal{P}$ is a family of parameter spaces;
- $\mathcal{I}$ is a collection of lawful interventions;
- $\mathcal{W}$ is a workload or experiment object;
- $\mathcal{E}$ is an evaluator, generally stochastic;
- $\mathcal{M}$ is a measurement algebra;
- $\mathcal{D}$ is a decision algebra or policy.

A concrete campaign adds a baseline state, finite evidence, resource budget, and proposal policy.

2.1 Required invariants

A useful backbone should guarantee or make checkable the following.

Typed closure. Only composable operations can be wired together.

Identity stability. Equal semantic plans have stable content identities under a canonical representation.

Effect visibility. Operations declare whether they are pure, CPU-bound, random, networked, or artifact-reading/writing. A plan can be inspected before it runs.

Dependency visibility. A parameter intervention can identify the closure of derived artifacts and evaluations that become invalid.

Intervention locality. A candidate that claims to change one component can be represented by a lawful local update rather than arbitrary mutation of an opaque blob.

Stochastic explicitness. Randomness is an input to evaluation semantics, and the coupling between baseline and candidate trials can be specified.

Failure preservation. Trial failure is a value in experimental evidence. It cannot silently disappear from aggregate comparison.

Decision separability. The mechanism for collecting evidence is distinct from the product policy that decides whether evidence is sufficient for promotion.

Resumability. Completed trial coordinates are durable; rerunning a campaign does not duplicate completed semantic work.

Plugin lawfulness. Extensibility is conditioned on schemas, descriptors, and laws, not only on implementation of an interface.

3. A Free Typed Process Category

3.1 Objects as ports

Let Σ be a set of schema identifiers. A wire carries a value of one schema. A port is a finite sequence

$$A = (\sigma_1, \dots, \sigma_n), \quad \sigma_i \in \Sigma.$$

The empty port $I = ()$ is the tensor unit. Concatenation defines the tensor product on objects:

$$A \otimes B = (\sigma_1, \dots, \sigma_m, \tau_1, \dots, \tau_n).$$

A primitive plugin operation declares an input port A and output port B . It is a generator

$$g : A \rightarrow B.$$

The kernel builds the free category generated by those operations plus structural wiring.

3.2 Composition

For morphisms $f : A \rightarrow B$ and $g : B \rightarrow C$, sequential composition is

$$g \circ f : A \rightarrow C.$$

For independent morphisms $f : A \rightarrow B$ and $g : C \rightarrow D$, tensor composition is

$$f \otimes g : A \otimes C \rightarrow B \otimes D.$$

Identity morphisms $\text{id}_A : A \rightarrow A$ are structural. Symmetries reorder wires. Copy and drop require care because a generic symmetric monoidal category does not give copying and deletion for free. The sandbox intentionally includes explicit structural Copy and Drop nodes for data values, making the plan language closer to a cartesian or gs-monoidal wiring syntax for ordinary immutable values. Their explicit presence is valuable: interpreters can assign cost, lineage, or restrictions to duplication.

The categorical laws are the expected ones:

$$(h \circ g) \circ f = h \circ (g \circ f),$$

$$\text{id}_B \circ f = f = f \circ \text{id}_A,$$

and tensor is associative and unital up to the chosen strict representation. A software implementation need not quotient syntax by all categorical equations. It can preserve a concrete syntax tree and provide canonicalization where identity matters.

3.3 Free syntax and interpreters

The crucial architectural move is to *not execute operations when they are composed*. Composition produces a plan value. A plan is syntax in the free category. An interpreter later assigns a meaning.

If Plan is the free category generated by plugin operations and $\mathcal{S}$ is a semantic category, an interpreter is a structure-preserving map

$$F : \text{Plan} \rightarrow \mathcal{S}.$$

For the runtime interpreter, $F(g)$ is an executable Go function. For a cost interpreter, $F(g)$ is a cost element. For an effect interpreter, $F(g)$ is an effect set. For provenance, it is a lineage transformer. The free syntax therefore makes analysis compositional.

This is the same architectural reason that query planners, compiler IRs, and algebraic effect systems retain syntax rather than immediately running callbacks.

3.4 Cost as a monoidal interpretation

Let a primitive descriptor carry

$$c(g) = (w, d, p),$$

where w is total work, d monetary cost, and p critical-path cost. Sequential and parallel composition use different algebras:

$$c(g \circ f) = (w_f + w_g, d_f + d_g, p_f + p_g),$$

while idealized parallel tensor uses

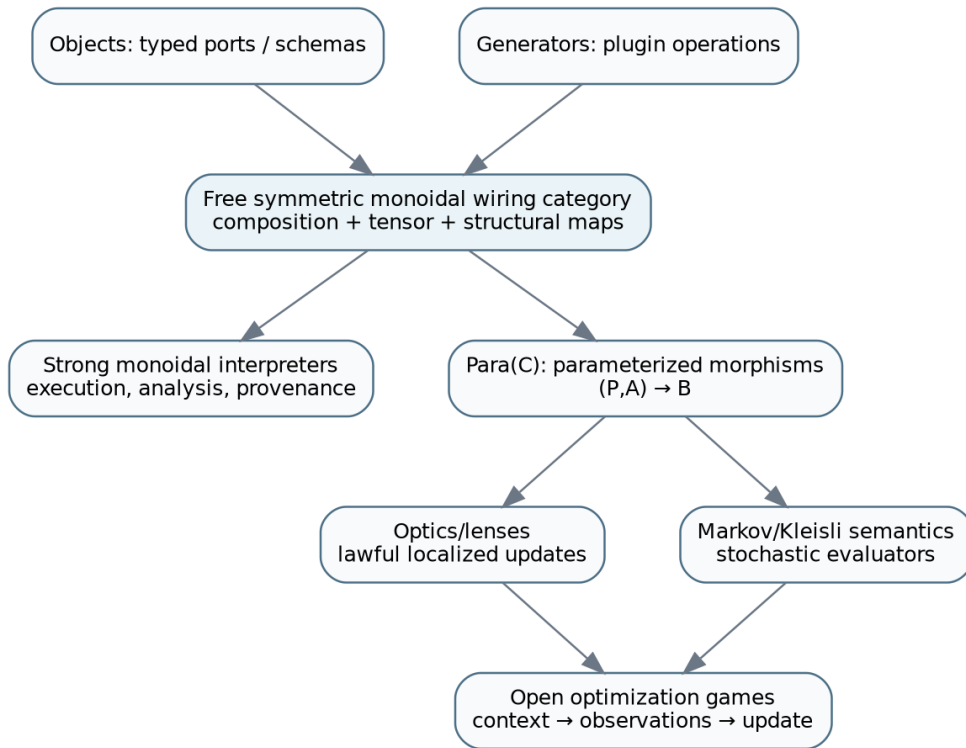


Figure 2: Categorical structures used by the architecture.

$$c(f \otimes g) = (w_f + w_g, d_f + d_g, \max(p_f, p_g)).$$

This simple example demonstrates why syntax matters. A flat list of operations cannot distinguish these meanings.

3.5 Effects as static annotations

Let E be a finite set of effect labels. Primitive operations declare effect sets $\epsilon(g) \subseteq E$. Composition combines by union:

$$\epsilon(g \circ f) = \epsilon(f) \cup \epsilon(g), \quad \epsilon(f \otimes g) = \epsilon(f) \cup \epsilon(g).$$

A policy interpreter can reject a plan before execution if, for example, a supposedly offline experiment contains network effects, or a protected-data plan includes a remote operation without a disclosure capability. The effect system need not be a full theorem prover to be useful.

3.6 Dependency labels

Primitive operations also declare semantic dependency labels such as `index.chunk`, `query.fusion`, or `eval.answer`. Static analysis of a plan forms their union. A candidate intervention declares target dependencies and a closure. A verifier can compare the declared closure to the plan’s dependencies and domain-specific dependency graph.

This converts invalidation from convention into data. The same mechanism can drive cache invalidation, build planning, or selection of required evaluation fidelities.

4. Plugins as Algebraic Signatures

4.1 Why conventional plugin callbacks are too weak

A common plugin API looks like:

```
type Plugin interface {
    Run(context.Context, any) (any, error)
}
```

This is flexible and nearly semantically empty. The host cannot know input/output types, side effects, cost, determinism, cacheability, or the dependencies of the computation. It cannot compose plugins safely without executing them. It cannot test plugin laws except through ad hoc integration tests.

The proposed plugin boundary is closer to an algebraic signature. A plugin contributes:

- schema identities;
- primitive operations with typed ports;
- operation descriptors;

- implementations of those operations;
- lawful intervention spaces or optics;
- conformance laws;
- optionally native trial runners and metric projections.

The registry validates uniqueness and executes plugin law suites at registration.

4.2 A plugin does not own composition

A key rule is that the plugin cannot decide how its operations compose with the rest of the system. It registers generators. The kernel owns the free composition constructors. This avoids a class of frameworks where each plugin secretly implements its own orchestration semantics.

Domain code can still offer convenience builders. A RAG plugin may expose `HybridRetrievePlan(...)`, but the returned value is ordinary kernel plan syntax. Static interpreters remain applicable.

4.3 Plugin laws

A law is not a substitute for proof, but it is stronger than an interface signature. Examples include:

- a lens satisfies Get-Put, Put-Get, and Put-Put;
- an operation marked deterministic returns equal canonical outputs for fixture inputs;
- a cacheable operation's declared semantic key changes when any material parameter changes;
- a metric projection never emits NaN or infinity;
- an evaluator preserves trial coordinates;
- a RAG evidence projection never promotes a derived representation to source authority.

A registry may reject a plugin when its law suite fails.

4.4 Capability-oriented extension

Some extensions should be optional capabilities rather than mandatory interfaces. For example:

```

Plugin
├─ operations
├─ laws
├─ optional CandidateSpace
├─ optional TrialRunner
├─ optional NativeArtifactResolver
└─ optional StaticVerifier

```

The kernel should discover these explicitly. It should not use reflection to infer semantic capabilities from method names.

4.5 Process isolation

The sandbox uses in-process Go interfaces for clarity. Production systems can preserve the same semantics across a process boundary. A remote plugin can publish a signed manifest containing schema and operation descriptors, then expose execution through RPC. The host still constructs and validates plans locally. Remote execution becomes one interpreter target. This permits language diversity and fault isolation without weakening the abstract model.

5. Parameterized Morphisms and the Para Construction

5.1 Systems vary with parameters

Optimization studies families of computations. A deterministic system can be written as

$$f : P \times A \rightarrow B,$$

where P is a parameter object, A an input, and B an output. Such a function is a parameterized morphism from A to B .

The categorical Para construction packages these morphisms so they compose while accumulating parameter objects. If

$$f : P \times A \rightarrow B \quad \text{and} \quad g : Q \times B \rightarrow C,$$

then their composite has parameter object $P \times Q$:

$$(g \star f) : (P \times Q) \times A \rightarrow C,$$

$$(g \star f)((p, q), a) = g(q, f(p, a)).$$

This is exactly what a multi-stage configurable pipeline requires. Chunking parameters and fusion parameters remain separate coordinates even when their operations compose.

5.2 Reparameterization

A map

$$r : R \rightarrow P$$

can reparameterize f into

$$f \circ (r \times \text{id}) : R \times A \rightarrow B.$$

In software this supports configuration compilation. A product may expose a high-level SearchProfile while the underlying operation expects low-level BM25, vector, and fusion settings. The high-level profile maps into the primitive parameter product without changing the underlying computation.

5.3 Optimization spaces are not necessarily Euclidean

Nothing in the construction requires parameters to be vectors. Parameter objects can be:

- finite enums;
- typed records;
- trees;
- immutable artifact references;
- prompts;
- index backend choices;
- policy expressions;
- complete release manifests.

This is important for RAG. Many useful interventions are structural or discrete. A mathematical architecture based only on gradients or $\mathbb{R}^n$ would exclude the most consequential design choices.

6. Optics and Lawful Interventions

6.1 Local change as a first-class object

Suppose a product configuration is S , and an optimizer wants to modify a component A . A lens consists of

$$\text{get} : S \rightarrow A$$

and

$$\text{put} : S \times A \rightarrow S.$$

The standard lens laws are:

Get-Put

$$\text{put}(s, \text{get}(s)) = s.$$

Put-Get

$$\text{get}(\text{put}(s, a)) = a.$$

Put-Put

$$\text{put}(\text{put}(s, a), b) = \text{put}(s, b).$$

These laws express what “this parameter” means. Without them, a setter may accidentally mutate unrelated fields or normalize the whole configuration in surprising ways.

6.2 Why optics matter to experimental validity

A candidate may claim to change only reranker pool size. If its materialization code also changes the prompt or model alias, the experiment is confounded. A lawful lens cannot by itself prove that external files are unchanged, but it gives the kernel an explicit local-update mechanism that can be combined with immutable snapshots and dependency checking.

The intervention becomes a value:

$$I = (\ell, a'),$$

where ℓ is the optic identity and a' the new focus value. Applying it to baseline s produces $s' = \text{put}_\ell(s, a')$.

The candidate artifact records baseline identity, optic identity, new value identity, and resulting snapshot identity. This is stronger than recording a shell command or a textual diff.

6.3 Composition of lenses

If $\ell_1 : S \rightsquigarrow A$ and $\ell_2 : A \rightsquigarrow B$, their composition focuses from S to B . This lets a generic optimizer target deeply nested domain configuration while the domain package owns all schema semantics.

For example:

```
ReleaseSpec
  .Query
  .Fusion
  .VectorWeight
```

is a composition of product-owned lenses. The core sees a lawful optic ID and typed focus schema, not field offsets or arbitrary JSON paths.

6.4 Beyond lenses

Lenses cover replaceable fields well. Other optimization structures may need:

- prisms for alternatives in a sum type;
- traversals for repeated homogeneous components;
- affine traversals for optional values;
- bidirectional transformations where normalization matters.

The kernel should not implement a giant optics library. It needs an intervention protocol that can carry an optic identity, focus type, and law suite. Domain packages can use richer optic constructions internally.

7. Probability Kernels and Experimental Semantics

7.1 Evaluation is a stochastic map

Even when a system is deterministic, a workload may be sampled. When a system contains model calls or measured latency, evaluation itself is stochastic.

A Markov kernel from X to Y maps each $x \in X$ to a probability distribution over Y :

$$K : X \rightsquigarrow Y.$$

For finite spaces, this is simply

$$K(x)(y) \geq 0, \quad \sum_y K(x)(y) = 1.$$

Kernels compose by integration/summation:

$$(L \circ K)(x)(z) = \sum_y K(x)(y)L(y)(z).$$

This is Kleisli composition for the distribution monad and, more abstractly, a basic Markov-category structure.

The importance for architecture is that stochasticity composes *semantically*. The evaluation of a candidate can be understood as a kernel even if the software obtains one sample at a time.

7.2 Deterministic seed splitting

A practical experiment engine does not enumerate distributions. It samples. Reproducibility therefore requires deterministic derivation of random sources from semantic coordinates:

$$\text{seed} = H(\text{suite}, \text{case}, \text{repeat}, \text{purpose}).$$

A split function derives independent sub-seeds by labeled hashing rather than mutable global RNG state.

This provides two useful properties.

1. Scheduling order does not change random streams.
2. Parallelism can be changed without changing semantic trial coordinates.

The accompanying sandbox implements deterministic seeds as SHA-256-derived values.

7.3 Coupling baseline and candidate

Comparing two stochastic systems requires more than independent samples. A *coupling* of distributions μ and ν is a joint distribution whose marginals are μ and ν . Paired experiments intentionally choose a coupling so nuisance variation is shared.

For baseline b , candidate c , case i , and repeat r , define shared seed

$$s_{i,r} = H(i, r).$$

Then evaluate

$$Y_b = E(b, i, s_{i,r}), \quad Y_c = E(c, i, s_{i,r}).$$

The paired difference

$$\Delta_{i,r} = m(Y_c) - m(Y_b)$$

has lower variance when the shared random source induces positive correlation in nuisance effects. The key point is conceptual: pairing is part of experiment semantics, not only a scheduling convenience.

7.4 Distributional equality and observational equivalence

Optimization frequently compares systems that need not have identical internal traces. Define an observation map

$$o : T \rightarrow O$$

from rich trial trace T to protected observable O . Two stochastic systems are observationally equivalent when the pushforward distributions agree:

$$o_* K_1(x) = o_* K_2(x).$$

In practice exact equality is rare. The architecture allows a product to define tolerances or noninferiority relations. An ANN index, for example, claims an approximation relation to an exact oracle rather than equality.

7.5 Why the generic kernel should not own statistical doctrine

The kernel should supply pairing, coordinates, retained trials, and metric vectors. It should not prescribe one statistical test. A deterministic retrieval suite, a stochastic judge, a production canary, and a latency benchmark have different sampling units and assumptions. Product or evaluation plugins should select bootstrap, permutation, sequential, Bayesian, or exact methods as appropriate.

8. Metrics, Ordered Decisions, and Open Games

8.1 Metrics form observations, not utility by default

Let a trial produce a metric vector

$$m(t) \in \mathbb{R}^k.$$

There is generally no canonical map $\mathbb{R}^k \rightarrow \mathbb{R}$ that represents product preference. Security violations, missing paired cells, latency, cost, answer quality, and relevance should not be silently commensurated.

The default comparison structure should therefore be a product partial order plus explicit gates.

For metrics with directions $d_j \in \{\min, \max\}$, candidate c weakly dominates b when it is no worse in every protected metric and strictly better in at least one preferred metric.

8.2 Ordered gates

A decision policy can be modeled as a sequence

$$G = (g_1, g_2, \dots, g_n)$$

where each gate consumes accumulated evidence and returns pass, fail, or insufficient. Evaluation stops at the first hard failure.

A RAG policy might be:

- coverage
 - authorization and disclosure
 - artifact / grounding integrity
 - protected retrieval noninferiority
 - answer quality improvement
 - latency and cost
 - preference among Pareto survivors

This is lexicographic rather than scalar optimization. It corresponds much more closely to real promotion policy.

8.3 Open-game intuition

Optimization has a feedback character similar to open games and compositional game theory. A component is not judged only by forward behavior. It is judged in a *context* that supplies observations or preferences from downstream.

The architecture does not require implementing a full category of open games. The useful intuition is:

- forward semantics produce outputs and traces;
- evaluation contexts turn those into observations;
- decision policies propagate acceptability information backward to candidate selection.

A plugin can therefore be composed as an executable component while leaving its optimization strategy open to the surrounding campaign.

9. Campaigns as Coalgebraic Feedback Processes

9.1 Campaign state

A campaign is not a single morphism because it evolves based on accumulated evidence. Let campaign state be

$$S = (b, C, E, B, q),$$

where b is baseline, C candidate frontier, E retained evidence, B remaining budget, and q campaign status.

One step computes either a terminal decision or a new state:

$$\gamma : S \rightarrow \text{Done}(D) + \text{Continue}(S).$$

This has a coalgebraic shape: state reveals the next observable action and successor state.



Figure 3: Campaign feedback structure.

9.2 Proposal strategies are plugins over evidence

The proposer need not be primitive to the kernel. A simple proposer enumerates finite patches. More advanced proposers can implement:

- grid or random search;
- successive halving;
- Bayesian optimization;
- evolutionary search;
- human-in-the-loop proposals;
- LLM-generated structured candidates;
- theorem-guided or constraint-solving proposals.

The proposer receives a read-only campaign view and returns typed intervention values. It cannot mutate run custody or mark trials complete. This is an important plugin-security boundary.

9.3 Trial coordinates

A trial is identified by semantic coordinate

$$(c, i, r, a),$$

for candidate, case, repeat, and arm. Durable evidence stores this coordinate explicitly. The campaign can resume by computing the set difference between required coordinates and committed coordinates.

A failed trial still occupies its coordinate. This prevents “retry until success then average successful trials,” which biases evaluation.

9.4 Append-only evidence

The simplest durable model is an append-only event log:

```
candidate_declared  
trial_committed  
comparison_written  
gate_evaluated  
decision_written
```

The sandbox implements JSONL trial and decision events. A production implementation should add canonical IDs, checksummed artifacts, expected event positions, terminal-run immutability, and explicit failure classes. The semantic principle is already visible: run state is reduced from durable evidence rather than maintained as a mutable bag of counters.

9.5 Termination

A campaign terminates when its policy reaches a terminal decision, its proposal frontier is exhausted, or budget is exhausted. These are different terminal classes. “No better candidate found” is not the same as “experiment incomplete.”

10. The Minimal Software Kernel

10.1 Kernel responsibilities

The proposed kernel is intentionally narrow.

Core values - schema IDs and typed envelopes; - canonical content digests; - operation descriptors; - explicit outcomes and observations.

Plan syntax - primitive operation; - identity; - sequence; - tensor; - copy/drop/permutation.

Plugin registry - operation registration; - descriptor lookup; - duplicate protection; - registration-time law checks.

Interpreters - runtime execution; - static effect/dependency/cost analysis; - plan identity.

Optimization structures - candidate spaces and interventions; - trial runner and suite; - deterministic seeds; - resumable campaign coordinates; - metric vectors and decisions.

Everything else is a plugin or a higher-level package.

10.2 What is intentionally absent

The kernel does *not* know:

- what RAG is;
- how to build an index;
- what a prompt is;
- how to call an LLM;
- what nDCG means;
- how to deploy a candidate;
- how many repeats are statistically sufficient;
- what “best” means for a product;
- what storage backend production should use;
- how a distributed workflow scheduler works.

Absence is a feature. These meanings vary by domain and product.

10.3 Typed envelopes and identity

The sandbox uses a typed envelope

```
type Envelope struct {  
    Schema SchemaID  
    Data    json.RawMessage  
    Digest  Digest  
}
```

with digest

$$H(\text{envelope/v1, schema, canonicalBytes}).$$

This is not a perfect universal canonicalization scheme, but it demonstrates domain-separated identity. Production should use a precisely specified canonical format with golden cross-language vectors.

10.4 Descriptors

A primitive descriptor states:

```
type Descriptor struct {  
    ID             OperationID  
    Plugin         string  
    Inputs         Port  
    Outputs        Port
```

```

    Effects      []Effect
    Dependencies []string
    Deterministic bool
    Cacheable    bool
    Cost          Cost
}

```

The descriptor is the semantic declaration visible to the host. Its implementation is intentionally separated.

10.5 Static inspection before execution

Because a plan is data, the host can answer:

- which plugins and operations are involved;
- which effects may occur;
- what dependency labels are touched;
- whether all primitives claim determinism;
- whether the composed plan is cacheable;
- estimated total work and critical path.

A production version can add capability requirements, trust zones, resource classes, data classifications, and artifact roles without changing the free-composition idea.

11. RAG as a Grafted Optimization Domain

11.1 The point of the example

The accompanying ragtoy package is intentionally small. It exists to prove a boundary: a RAG domain can register retrieval operations, parameter spaces, laws, cases, and a trial runner without introducing RAG-specific concepts into the generic kernel.

11.2 Domain types

The toy plugin defines schemas for:

- retrieval specification;
- corpus;
- query/relevance case;
- ranked result.

Its specification contains lexical weight, semantic weight, top-k, and a chunk-related parameter. The retrieval implementation scores a tiny corpus with lexical overlap and a deliberately simple semantic synonym map. This is not intended as a serious retriever. It makes the candidate/evaluation semantics visible without requiring external models or services.

11.3 The RAG operation as a primitive generator

The plugin registers

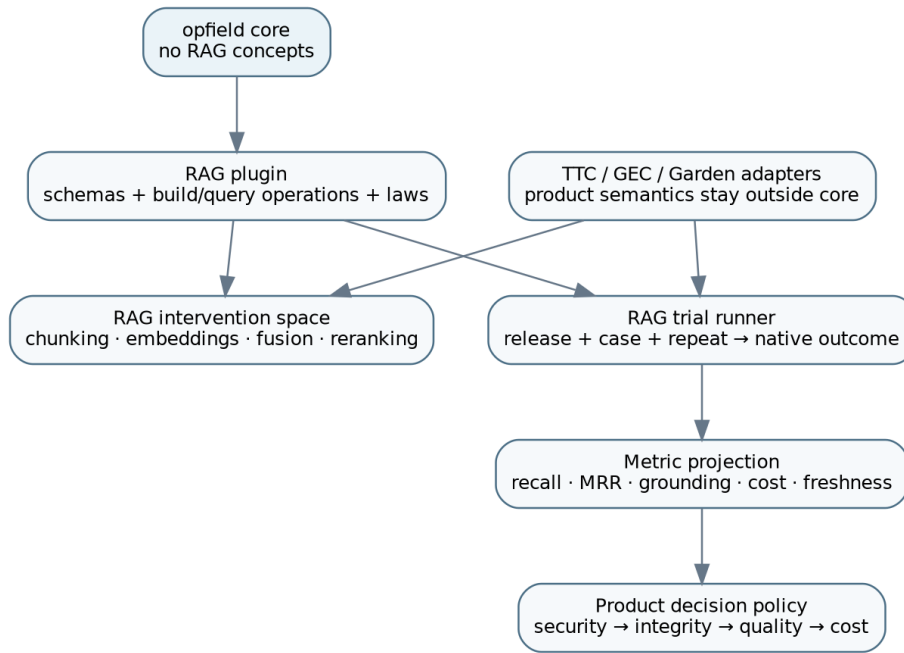


Figure 4: RAG grafted onto the generic optimization kernel.

retrieve : Corpus $\otimes$ Spec $\otimes$ Case $\rightarrow$ Result.

Its descriptor declares CPU effect, dependencies such as `index.chunk` and `query.fusion`, determinism, cacheability, and cost.

A more realistic RAG plugin would register separate operations:

```

normalize
chunk
represent
embed
lexical-index
vector-index
rewrite
lexical-search
vector-search
collapse
fuse
authorize
rerank
admit-evidence
generate
validate

```

These remain ordinary generators in the plan category. Product-level query inter-

preters compose them into plans.

11.4 Candidate spaces

The toy candidate space exposes different semantic-weight values. A production RAG candidate space would expose typed optics into a behavior-complete release specification. Examples:

$$\ell_{chunkSize} : R \rightsquigarrow \mathbb{N},$$

$$\ell_{embedding} : R \rightsquigarrow \text{ModelRef},$$

$$\ell_{vectorWeight} : R \rightsquigarrow \mathbb{R}_{\geq 0},$$

$$\ell_{reranker} : R \rightsquigarrow \text{RerankerSpec}.$$

Each optic has a dependency closure. Chunking affects derived index material. Fusion weight affects only query policy and downstream observations. The generic campaign engine does not need to know why.

11.5 Evaluation

The toy runner evaluates mean reciprocal rank and a simple utility. A production runner should retain a native artifact and project metrics such as:

- recall, MRR, nDCG;
- grounding and citation validity;
- answer quality;
- unauthorized disclosure count;
- latency and provider calls;
- cost;
- freshness;
- session success;
- frontend projection validity.

The metric projection is domain-owned. Core campaign custody merely preserves coordinates and values.

11.6 From ragopt to the proposed backbone

The existing ragopt architecture already contains several pieces of the campaign layer: immutable candidates, paired evaluation, durable run custody, comparison, gates, and reports. The theory here suggests how to place a more principled layer underneath it.

A migration can be incremental:

1. represent RAG build/query workflows as typed plans;
2. give operations descriptors with dependencies and effects;
3. represent candidate mutation through lawful optics into release specs;
4. let ragopt continue to own run/evidence custody;
5. adapt its arms to execute typed plans and retain native artifacts;
6. move generic plan/effect/optic/probability pieces into a small domain-neutral package only when two domains use them.

There is no requirement that the package be named `opfield`. It is the semantic boundary that matters.

12. Plugin Architecture for Real Systems

12.1 Four plugin strata

A practical implementation benefits from four distinct plugin strata.

Operation plugins

They register typed primitive computations. Examples: a specific embedding provider, vector backend, compiler invocation, or simulation step.

Domain plugins

They register schemas, lawful candidate optics, dependency graphs, native outcome types, and domain law suites. Example: RAG.

Evaluation plugins

They define suites, judges, metric projections, confidence procedures, and fidelity relations. Example: a Garden conversation evaluator.

Proposal plugins

They consume read-only campaign evidence and emit candidate interventions. Example: Bayesian optimization or LLM-guided proposal.

Keeping these strata separate prevents a provider adapter from becoming a campaign engine or a proposal algorithm from taking control of run storage.

12.2 Interface sketch

A production interface family might look like:

```
type OperationPlugin interface {
    Manifest() Manifest
    Operations() []Operation
    Laws() []Law
}
```

```

type DomainSpace[S any] interface {
    Baseline(context.Context) (S, error)
    Interventions(context.Context, S) ([]Intervention, error)
    Apply(context.Context, S, Intervention) (S, error)
    DependencyClosure(Intervention) Closure
}

type Evaluator[S any] interface {
    RequiredFidelity(Intervention) Fidelity
    Run(context.Context, TrialRequest[S]) NativeTrial
    Project(NativeTrial) MetricVector
}

type Proposer interface {
    Propose(context.Context, CampaignView) ([]Intervention, error)
}

```

The generic runtime may erase concrete types at durable boundaries using schema-tagged envelopes, while domain packages retain static types internally.

12.3 Manifested semantics

A plugin manifest should contain more than version strings. Useful fields include:

- plugin ID and semantic version;
- schema IDs and compatibility rules;
- operation descriptors;
- law-suite identity;
- implementation digest;
- required host capabilities;
- declared trust zone;
- deterministic/reproducibility class;
- data classes permitted for remote operations.

The manifest itself can be content-addressed and attached to every plan/release that uses the plugin.

12.4 Host control

The host must retain control over:

- composition;
- resource budgets;
- effect policy;
- artifact custody;
- trial coordinates;
- cancellation;
- run finalization;
- promotion authority.

A plugin should never be allowed to mark itself as having passed a gate or to mutate the active production release simply because it implements an evaluator.

13. Correctness and Proof Obligations

13.1 Composition laws

The free plan constructors should satisfy property tests for associativity and identity modulo chosen canonicalization. Tensor should preserve input/output concatenation. Permutations must be bijections. Copy and drop must have declared value semantics.

13.2 Interpreter coherence

Different interpreters should agree on structural meaning. For example, plan validation says a composition is well-typed; runtime execution must consume and produce exactly those ports. Static operation lists must contain every primitive actually executed.

A useful theorem schema is *sound static analysis*:

If static analysis of plan p says effect e is absent, then no conforming execution interpreter for registered primitives may perform effect e .

This requires plugin operation implementations to be constrained by descriptors, which is difficult to prove in ordinary Go. Process sandboxing, capability APIs, or effect-specific host services can make the claim enforceable rather than conventional.

13.3 Lens laws

Every intervention optic runs a law suite over representative/generated states. For pure record lenses, these laws are straightforward and highly valuable.

13.4 Seed and pairing laws

Seed splitting should be deterministic and label-sensitive. Trial seed must be independent of execution order. Baseline/candidate trials intended to be paired must derive from the same pair seed and use distinct purpose sub-seeds only where semantics requires it.

13.5 Evidence laws

For required coordinate set R and committed coordinate set C :

$$\text{pending} = R \setminus C.$$

Appending a duplicate terminal coordinate should either be rejected or be idempotent with exact value equality. A terminal campaign report may reference only committed trial artifacts. Failed trials remain in C .

13.6 Decision laws

Hard gates are ordered. If gate g_i fails, adding evidence relevant only to later preference gate g_j , $j > i$, cannot make the candidate eligible without changing the failed gate evidence. This monotonicity is testable.

14. Relationship to Existing Mathematical Work

This architecture draws from several lines of research without requiring the implementation to reproduce their full formalisms.

14.1 Monads and effectful computation

Moggi’s categorical semantics of computation separates pure values from effectful computation. In the present architecture, explicit effect descriptors and stochastic kernels use the same conceptual separation. Production interpreters could deepen this into typed capability/effect algebras.

14.2 Arrows and generalized computation

Arrows and related abstractions are relevant because optimization plans may be statically inspectable even when they are not monadic programs assembled dynamically. The free plan syntax is intentionally first-order and analyzable.

14.3 Parameterized maps and learning

Work on categorical formulations of learning emphasizes parameterized morphisms and compositional update structures. The Para construction directly motivates representing a pipeline as composition of independently parameterized stages rather than flattening all parameters into one vector.

14.4 Optics

Optics provide compositional access and update into structured state. For optimization they give a disciplined semantics for interventions, particularly in heterogeneous configuration trees.

14.5 Markov categories

Markov categories abstract stochastic maps and conditional independence. The thesis uses only a small fragment: kernels, deterministic maps, products, and couplings. This is enough to clarify the semantics of repeated and paired evaluation.

14.6 Open games and compositional optimization

Open games and compositional game theory formalize systems whose local behavior is evaluated in larger contexts. The optimization campaign here has a similar bidi-

rectional flavor. Rather than adopting the full apparatus, the architecture retains separate forward plans, observation maps, and decision policies.

14.7 Differentiable programming

Reverse differential categories and categorical treatments of backpropagation are important when components are differentiable and gradients are the update mechanism. They are not the universal backbone proposed here because most RAG interventions are discrete, structural, stochastic, or policy-valued. Gradient-based optimizers can be implemented as specialized proposal/update plugins over differentiable subspaces.

15. A More General Theory of Optimization Morphisms

15.1 Optimization as an outer category

One can distinguish the category of systems $\mathcal{C}$ from an outer category whose morphisms are transformations of parameterized systems. Objects are systems or release specifications; an optimization morphism carries an intervention plus evidence relation.

A simple intervention arrow is

$$I : S \rightarrow S'.$$

But an optimization step should also record why the step is admissible:

$$I : S \xrightarrow{E,D} S',$$

where E is evidence and D a decision certificate. Composition of such arrows concatenates evidence and requires the target of one step to be the baseline identity of the next.

This suggests an immutable optimization-history category in which a path is a lineage of accepted releases. Rejected candidates are not arrows in the accepted-history category; they remain experimental evidence in a larger trial graph.

15.2 Fibred view of parameter spaces

Different system designs have different parameter spaces. If a candidate changes index backend, the valid parameters for the backend change as well. A single global product space is unnatural.

A fibred perspective is useful: over each structural design s sits a fiber P_s of valid local parameters. Structural interventions move between fibers. Local tuning remains within a fiber. In implementation, this corresponds to typed sum types and plugin-specific spaces rather than one map of parameter names.

15.3 Multi-fidelity evaluation as morphisms between observation spaces

Let $O_0, O_1, \dots, O_n$ be increasing evaluation fidelities: static laws, retrieval metrics, answer tests, session calibration, load tests, shadow traffic, canary. There are projections from richer observations to coarser ones, but not generally inverses.

A candidate's intervention class determines the minimum observation space capable of validating its claims. This can be represented as a monotone requirement map

$$\phi : \text{InterventionClass} \rightarrow \text{Fidelity}.$$

The campaign allocator should not compare candidates using an observation space too weak to observe their semantic differences.

16. From Mathematical Core to Production Architecture

16.1 The core should remain library-sized

The danger of mathematically motivated architecture is framework inflation. The mathematical structures should *reduce* the core.

A reasonable production core might contain fewer than a dozen concepts:

1. schema identity;
2. typed immutable value/envelope;
3. operation descriptor;
4. plan constructors;
5. plan interpreters;
6. optic/intervention protocol;
7. trial coordinate/outcome;
8. seed/coupling protocol;
9. plugin manifest and law interface;
10. append-only evidence references.

Candidate algorithms, statistics, product gates, deployment, and domain types belong elsewhere.

16.2 Suggested package layering

```
opfield/  
  core/      identities, typed values, outcomes  
  plan/      free typed process syntax  
  plugin/    manifests, generators, law registry  
  engine/    execution interpreter  
  optic/     intervention protocol and law helpers  
  prob/      seeds, finite/reference kernels, coupling helpers  
  experiment/ trial coordinates, suites, spaces, native outcomes  
  campaign/  resumable orchestration and evidence reduction
```

```

ragspace/
  release/      behavior-complete RAG spec
  operations/   RAG primitive generators
  optics/       chunking, embedding, fusion, reranking interventions
  eval/         RAG-native trial runners and projections

products/
  gec/
  ttc/
  garden/

```

The exact repository boundary can differ. The dependency direction is the important part.

16.3 Relationship to ragkit and ragopt

One possible integration is:

```

opfield core
  ↑       ↑
ragkit   ragopt
  ↑       ↑
└─ product optimization adapters ─┘

```

ragkit uses the typed plan/effect/identity pieces for build/query semantics. ragopt uses trial, evidence, pairing, and candidate semantics. Product adapters combine them. Neither common package needs to import the other if the shared kernel remains genuinely domain-neutral.

A less fragmented alternative is to place the small kernel inside ragopt/internal/semantic until a second non-RAG domain demonstrates reuse. The sandbox's quadratic example would be a candidate proof. Package extraction should follow evidence, not aesthetic symmetry.

17. Sandbox Implementation

17.1 Scope

The accompanying opfield repository is self-contained and uses only the Go standard library. It demonstrates the core ideas rather than production completeness.

The repository contains:

```

core/      typed envelopes, digests, effects, descriptors
plan/      free plan syntax and static analysis
plugin/    registry, operations, laws
engine/    runtime plan interpreter
optic/     lawful lenses
prob/      deterministic seeds and finite distributions

```

```
experiment/ spaces, cases, trials
campaign/   append-only resumable campaign
ragtoy/     domain plugin
cmd/        runnable demonstration

go test ./... passes in the supplied environment.
```

17.2 Running the demo

```
go test ./...
go run ./cmd/opfield-demo -out ./demo-out
cat demo-out/plan.json
cat demo-out/result.json
```

Running again against the same output directory reads committed trial events and skips completed coordinates.

17.3 What the demo proves

The demo proves several architectural properties in executable form:

- the generic core imports no RAG package;
- the RAG plugin can register typed operations and laws;
- plans can be statically analyzed before execution;
- the same runtime can execute plugin operations;
- interventions are represented independently from evaluation;
- paired repeat coordinates use deterministic seed derivation;
- campaign evidence is append-only and resumable;
- RAG-specific metrics and utility remain in the RAG domain.

It does not prove production properties such as distributed exactly-once custody, cross-language canonicalization, provider security, or statistically valid real RAG evaluation. Those are intentionally left as next-layer contracts.

18. Worked RAG Mapping

Consider a production RAG release

$$R = (C, N, K, P, E, L, V, Q, F, A),$$

where C is corpus snapshot, N normalization, K chunking, P representations, E embedding, L lexical index, V vector index, Q query policy, F answer policy, and A auxiliary product assets.

A query plan might be:

rewrite; (lexical $\otimes$ vector); collapse; fuse; authorize; rerank; admit; generate; validate.

Each primitive has typed ports, effects, dependencies, and capability requirements.

Fusion intervention

A lens focuses Q .vectorWeight. Dependency closure:

```
query.fusion
retrieval.outcome
answer.outcome
session.outcome
```

No index artifact is invalidated.

Chunking intervention

A lens focuses K . Closure:

```
chunk
representation
embedding
lexical.index
vector.index
retrieval.outcome
answer.outcome
session.outcome
presentation.citations
```

The campaign can therefore request a rebuild evaluator rather than a query-only evaluator.

ANN intervention

A prism selects vector backend Exact or HNSW(config). The candidate changes approximation semantics and operational cost. Static laws require metric compatibility and filter capability. Evaluation compares the ANN kernel to an exact oracle under a declared recall relation and then propagates the candidate through answer-level trials.

This example illustrates the central value of the backbone: different optimization cases are grafted onto the same composition, intervention, stochastic-evaluation, and evidence machinery without pretending they are the same kind of change.

19. Research Agenda

The sandbox suggests several directions for doctoral-level refinement.

19.1 Canonical quotient of plans

The current plan syntax distinguishes trees that are categorically equivalent. A research implementation can define a canonical string-diagram representation or nor-

malization procedure so plan identity respects associativity, unit, and symmetry laws where appropriate.

19.2 Effect capabilities rather than labels

Static effect labels are descriptive. A stronger system gives each effect an unforgeable capability supplied by the host. An operation that declares network can perform network I/O only through a host capability. This would make effect soundness enforceable.

19.3 Enriched cost categories

Costs are not merely scalars. Latency distributions, resource vectors, privacy budgets, and monetary costs can form enriched hom-objects. Composition then uses domain-specific monoids or quantales. This can support preflight optimization of workflow structure itself.

19.4 Markov-category evaluation

The finite distribution reference package can be extended into a sampled-kernel interface with retained probability-model identities, conditional independence declarations, and coupling constructors. This would give stronger semantics to multi-stage stochastic RAG evaluation.

19.5 Optic-indexed dependency analysis

Each optic can carry a formal dependency morphism saying which plan generators may change under an intervention. Comparing this to static plan analysis would permit automatic invalidation proofs for large classes of candidates.

19.6 Compositional decision theory

Open games, selection functions, and preference relations may provide a stronger model for local optimizers composed into a global campaign. This is especially relevant when different subsystems have different objectives and constraints.

19.7 Differentiable subdomains

Some components, such as linear fusion weights or learned rerankers, support gradients. A differentiable plugin could expose a reverse derivative or learner structure while remaining embedded in the same broader discrete optimization field. The generic kernel need not choose between gradient and black-box optimization.

19.8 Formal verification

The most promising machine-checked targets are:

- plan typing and canonicalization;

- optic laws;
- seed/coupling invariants;
- evidence reducer safety;
- gate ordering;
- static effect-policy soundness under capability-based execution.

20. Conclusion

The optimization problem in complex software is not fundamentally “find the maximum of a function.” It is “change a structured system, realize the consequences, observe them under uncertainty, preserve evidence, and decide under constraints.” The mathematical object is therefore richer than a numeric search space.

The architecture developed here starts with a small process category. Plugins provide typed generators. Free composition produces inspectable plans. Parameterized morphisms explain how configurable stages compose. Optics make interventions local and lawful. Probability kernels make stochastic evaluation explicit. Couplings explain paired comparison. Metric algebras and ordered gates keep product preference outside the generic engine. Campaigns form resumable feedback state machines over immutable evidence.

This structure creates the desired extensibility boundary. The core is simple because it does not know domain meaning. The semantics are strong because plugins cannot bypass typing, descriptors, laws, coordinates, and evidence custody. RAG can graft indexing and querying operations, candidate spaces, evaluators, and promotion policy onto the backbone. A compiler tuner, scheduler optimizer, or simulation system can graft different operations onto exactly the same abstract machinery.

The most important design recommendation is therefore not to build a universal “optimization framework.” Build a small algebra of typed processes and interventions, then let domains supply models of that algebra. Optimization becomes compositional because the *meaning of composition* is centralized, while the meaning of each domain remains local.

Appendix A. Core Algebra

For reference, the minimal algebra is summarized below.

Objects:

$$A, B, C ::= I \mid \sigma \mid A \otimes B.$$

Morphisms:

$$f, g ::= \text{id}_A \mid \text{op}_i \mid g \circ f \mid f \otimes g \mid \text{copy}_A \mid \text{drop}_A \mid \text{permute}.$$

Parameterized morphism:

$$f : P \otimes A \rightarrow B.$$

Lens intervention:

$$\ell : S \rightsquigarrow A = (\text{get}, \text{put}).$$

Stochastic evaluator:

$$E : \text{Candidate} \otimes \text{Case} \otimes \text{Seed} \rightsquigarrow \text{Trial}.$$

Metric projection:

$$m : \text{Trial} \rightarrow \mathbb{R}^k + \text{Failure}.$$

Campaign step:

$$\gamma : S \rightarrow \text{Done}(D) + \text{Continue}(S).$$

Appendix B. Plugin Checklist

A domain plugin should answer:

1. What schemas does it introduce?
2. Which primitive operations does it introduce?
3. What are each operation's typed ports?
4. Which effects can each operation perform?
5. Which semantic dependencies does each operation touch?
6. Is it deterministic? Under what identity?
7. Is its output cacheable? Under what semantic key?
8. What laws does it claim?
9. What intervention optics are legal?
10. What dependency closure follows from each intervention?
11. What evaluation fidelity can observe each intervention class?
12. What native artifact must be retained for diagnosis?
13. Which metric projections are comparable across candidates?
14. Which failures are ordinary trial outcomes versus infrastructure failures?
15. Which product decision policy ultimately governs promotion?

Appendix C. Sandbox File Map

```
opfield/
├─ README.md
├─ go.mod
├─ core/
```

```

|   |   | core.go
|   |   | core_test.go
|   |   |
|   |   | plan/
|   |   | |   | plan.go
|   |   | |   | plan_test.go
|   |   | |   |
|   |   | |   | plugin/
|   |   | |   | |   | plugin.go
|   |   | |   |
|   |   | |   | engine/
|   |   | |   | |   | executor.go
|   |   | |   |
|   |   | |   | optic/
|   |   | |   | |   | lens.go
|   |   | |   |
|   |   | |   | prob/
|   |   | |   | |   | prob.go
|   |   | |   | |   | prob_test.go
|   |   | |   |
|   |   | |   | experiment/
|   |   | |   | |   | experiment.go
|   |   | |   |
|   |   | |   | campaign/
|   |   | |   | |   | campaign.go
|   |   | |   |
|   |   | |   | domain/ragtoy/
|   |   | |   | |   | ragtoy.go
|   |   | |   | |   | ragtoy_test.go
|   |   | |   |
|   |   | |   | cmd/opfield-demo/
|   |   | |   | |   | main.go

```

Appendix D. Suggested Next Implementation Increments

1. Replace JSON canonicalization with a published canonical codec and golden vectors.
2. Add operation-manifest identity to plan identity.
3. Add a structural fold so all plan interpreters are explicitly derived from one recursion scheme.
4. Add a capability-based effect runtime.
5. Add artifact references and immutable native trial artifacts.
6. Distinguish trial failure, infrastructure failure, and cancellation.
7. Add exact incumbent/challenger paired coordinates rather than candidate-only aggregates.
8. Add metric definitions with direction and missingness semantics.
9. Add ordered gate interfaces and Pareto reporting.
10. Add a behavior-complete RAG release plugin using ragkit types.
11. Wrap current ragopt run custody around the new candidate/plan semantics rather than replacing it wholesale.
12. Add a second non-RAG domain before extracting a permanent cross-project package.

Appendix E. Selected Bibliography

- Moggi, Eugenio. “Notions of Computation and Monads.” *Information and Computation* 93(1), 1991.
- Mac Lane, Saunders. *Categories for the Working Mathematician*. Springer.
- Fong, Brendan, David Spivak, and Rémy Tuyéras. “Backprop as Functor: A Compositional Perspective on Supervised Learning.”
- Cruttwell, G. S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. Work on categorical foundations of gradient-based learning and parameterized maps.
- Riley, Mitchell. “Categories of Optics.”
- Fritz, Tobias. “A Synthetic Approach to Markov Kernels, Conditional Independence and theorems on Sufficient Statistics.” *Advances in Mathematics*, 2020.
- Fong, Brendan, and David Spivak. *An Invitation to Applied Category Theory*. Cambridge University Press, 2019.
- Hedges, Jules. Work on compositional game theory and open games.
- Plotkin, Gordon, and Matija Pretnar. Work on algebraic effects and handlers.
- Claessen, Koen, and John Hughes. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” ICFP 2000.
- Lewis, Patrick et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS 2020.
- Karpukhin, Vladimir et al. “Dense Passage Retrieval for Open-Domain Question Answering.” EMNLP 2020.